



IIC2333 — Sistemas Operativos y Redes — 1/2024
Ayudantía Midterm

1. Procesos y Syscall

A partir del siguiente código, responda las siguientes preguntas.

```
1 void handle_signal(int signal) {  
2     if (signal == SIGALRM) {  
3         printf("Captured Alarm\n");  
4         kill(0, SIGKILL)  
5     }  
6 }  
7  
8 int main(int argc, char *argv[]) {  
9     pid_t pids[5];  
10  
11     signal(SIGALRM, handle_signal);  
12     alarm(4);  
13  
14     int i;  
15     for (i = 0; i < 5; i++) {  
16         pids[i] = fork();  
17         if (pids[i] == 0) {  
18             sleep(i + 1);  
19             execl("/bin/ls", "/bin/ls", NULL);  
20         }  
21     }  
22  
23     for (i = 0; i < 10; i++) {  
24         waitpid(pids[i], NULL, 0);  
25     }  
26     return 0;  
27 }
```

1. Identifique y explique para que sirven las syscalls vistas en clases y las señales involucradas.

R: fork: Genera un duplicado del proceso actual, es decir, es una copia del proceso padre. La función crea un proceso hijo a partir del proceso padre y devuelve el PID (identificador de proceso) del hijo.

waitpid: Se utiliza para que el proceso padre espere a que sus procesos hijos terminen su ejecución. Esta syscall se encuentra en el segundo bucle `for` y espera a que cada uno de los procesos hijos termine para continuar con la ejecución del proceso padre.

exec (execl): Reemplaza el proceso por otro y comúnmente utilizada por los programas para ejecutar otros programas en el sistema operativo. Además, incluye varias funciones con diferentes nombres y argumentos, en el código se encuentra `execl()`.

kill: enviar señales. SIGKILL: Es una señal que se envía a un proceso para que termine de manera inmediata.

SIGALRM: Es una señal que se envía cuando el timer asociado a la función `alarm` expira.

2. Se obtiene el siguiente output, ¿por qué aparece *Killed*?

```
SO Hello_World.txt process.c REDES
SO Hello_World.txt process.c REDES
SO Hello_World.txt process.c REDES
Captured Alarm
Killed
```

R: El mensaje “Killed” en la salida se debe a la presencia de la función `handle_signal` que maneja la señal `SIGALRM`. Por lo tanto, una vez cumplido los 4 segundos, se activa el manejador de señales y se envía `SIGKILL`. Esto provoca que se propague la señal al grupo de procesos y se imprima “Killed”.

3. Considerando el flujo del programa y el output de la pregunta anterior, ¿cuántos hijos se lograron crear?

R: Se lograron crear 5 hijos, lo podemos evidenciar por el `for`.

2. Scheduling - Midterm 2023-1

1. Se tiene un nuevo scheduler y le piden que lo ponga a prueba. Las características del scheduler son las siguientes:

- Scheduler cuenta con 2 colas, una de prioridad alta (cola `High`) y otra de prioridad baja (cola `Low`). Los procesos en la cola `High` siempre tienen prioridad sobre la cola `Low`.
- Los procesos entran al sistema en estado `READY` y en la cola de prioridad alta (cola `High`).
- El scheduler usa un sistema Round-Robin en ambas colas de `quantum = 2`.
- Un proceso sale de la CPU si la cede o si se acaba su `quantum`. Si estos pasan en el mismo tiempo, entonces se considera que el proceso fue interrumpido.
- Si un proceso en cola de prioridad alta (cola `High`) termina su `quantum`, baja a una cola de prioridad baja (cola `Low`). Si esta en una cola de prioridad baja (cola `Low`), se mantiene en esta y se reinicia su `quantum`.
- Los procesos solo reinician su `quantum` al cambiar de cola de prioridad (de cola `High` a cola `Low` o `Low` a `High`).
- La prioridad dentro de una cola está determinada por FIFO (FCFS).
- Si un proceso pasa más de 3 segundos desde la última vez que salió de la CPU y está en la cola `Low`, se debe subir al final de la cola `High`.

Al scheduler se le asignan los siguientes procesos:

Nombre	Tiempo Ingreso	Total CPU Use	CPU Burst	I/O Wait
process1	1	5	3	3
process2	8	2	2	0
process3	0	5	2	2

Tiene la siguiente tabla con información incompleta del scheduler:

Nombre	0	1	2	3	4	5	6	7
process1		Re/H	Ru	Ru	Re/L	Re/L	Ru	W/L
process2								
process3	Ru	Ru	W/L	W/L	Ru	Ru	W/L	W/L

H= Cola High; L= Cola Low; Ru= Running; Re= Ready; W=I/O wait.

1.1) Indique el estado y la cola en que se encuentran los procesos en el segundo 10.

Process1: Estado= Ready; Cola= High Process2: Estado= Finished; Cola= None (importante no poner cola!) Process3: Estado Running; Cola= None (importante no poner cola!)

1.2) Calcule el *turnaround time* promedio de los procesos.

Process1: 12 Process2: 2 Process3: 11 Promedio: 8.3333333

3. Sincronización - Ayudantía 2023-2

Se presenta un problema en donde se requiere que dos threads modifiquen dos variables compartidas, sin embargo, los threads no deben acceder al mismo tiempo a una variable. Para esto, se presenta la siguiente solución usando semáforos binarios, y le piden a usted que la evalúe:

```
1  // Variables compartidas
2  struct lock l;
3  int A = 0;
4  int B = 0;
5  struct semaphore disponibleA; disponibleA.init(1);
6  struct semaphore disponibleB; disponibleB.init(1);
7
8  // Thread 1
9  while(true) {
10     disponibleA.P();
11     A += 1;
12     disponibleB.P();
13     B += 1;
14     disponibleB.V();
15     disponibleA.V();
16 }
17
18 // Thread 2
19 while(true)
20 {
21     disponibleB.P();
22     B += 2;
23     disponibleA.P();
24     A += 2;
25     disponibleA.V();
26     disponibleB.V();
27 }
```

A partir de este código responda las siguientes preguntas

3.1. ¿Que secciones críticas existen el código? (Indique líneas)

R: Las líneas 11, 13, 22 y 24 se modifican variables compartidas, por lo tanto, debe ser consideradas dentro de una sección crítica.

3.2. ¿Cómo se llaman las propiedades que deben cumplir las secciones críticas del código? ¿Las cumple? Justifique.

R:

Las propiedades son: exclusión mutua, progreso y espera acotada.

La primera, exclusión mutua, sí lo cumple, esto porque con los semáforos protegemos las secciones críticas. Si un thread disminuye el contador de un semáforo (llama a P()) y entra a la sección crítica, y luego otro thread quiere

disminuir nuevamente ese contador, no lo podrá realizar y tendrá que esperar hasta que el primer thread que entró a la SC lo aumente (llame a V()).

Progreso no se cumple, debido a que se puede presentar la situación en que un thread disminuya el contador del semáforo disponibleA (línea 10), modifique la variable A (línea 11) y al mismo tiempo, un segundo thread disminuya el contador de disponibleB (línea 23) y modifique la variable B (línea 24). El problema está en que luego el primer thread intentará disminuir el contador de disponibleB (línea 12), mientras que el segundo thread intentará disminuir el contador de disponibleA (línea 25). Ambos no podrán realizarlo y se bloquearán a espera de que otro thread aumente el contador del semáforo correspondiente, sin embargo, este aumento solo lo pueden realizar los threads que están espera, llegando a una situación en donde no se avanza. Luego, como ambos threads quieren entrar a secciones críticas que están disponibles y no pueden, entonces no se cumple progreso.

Por último, y tomando la situación anterior, vemos que es posible que los threads deban esperar un tiempo infinito para entrar a la SC, por lo tanto, no se cumple espera acotada.

4. Sistemas de Archivo

4.1 Para que el funcionamiento de un disco duro tradicional sea óptimo, es de suma importancia defragmentar el disco periódicamente, para que los bloques que se suelen leer en conjunto (como los bloques de datos correspondientes a un mismo archivo) estén cercanos, de forma de minimizar los tiempos de búsqueda en el disco físico. ¿Esto también es cierto para un SSD?

R: No, ya que un SSD funciona más parecido a una memoria RAM, por lo que no tiene tiempos de búsqueda física asociados.

4.2 ¿Qué necesita un sistema de archivos para que tenga soporte para *soft links*? ¿Y para *hard links*?

R: Un *soft link* no es más que un archivo de texto con un path, así que su implementación no requiere cambios en el sistema. Por otro lado, los *hard links* requieren almacenar al menos la cantidad de referencias que tiene hacia él para que, cuando este número baje a 0, se pueda eliminar el archivo y liberar los bloques asociados a él en el bitmap.

4.3 V o F: Una ventaja de los *soft links* por sobre los *hard links* es que los *soft links* pueden apuntar a directorios.

R: V. Un *hard link* no puede apuntar a un directorio porque esto provocaría un ciclo en el árbol de directorios (y no sería un árbol).

4.4 En un sistema de archivos determinado se está intentando crear un nuevo archivo vacío en un directorio existente, por lo que es necesario modificar tres bloques: el bitmap, el bloque de directorio y el bloque índice del archivo. Durante este proceso hay una caída del sistema y no se alcanzan a actualizar todos los bloques. Asuma que no se ejecutan herramientas de verificación de consistencia como FSCK (Unix) o chkdsk (Windows). Dadas las siguientes posibles consecuencias (considere que estas se pueden presentar inmediatamente o en un futuro a causa de esta caída):

- (a) No hay ni habrá inconsistencias (parecerá como si la operación simplemente no se hubiera hecho)
- (b) Hay un *data leak* (similar a un *memory leak*, pero para disco)
- (c) Con distintos *file paths* se podría llegar al mismo bloque índice
- (d) Hay punteros que apuntan a cualquier cosa
- (e) Varios problemas de los antes mencionados

Cuál de estas es la que corresponde si **solamente** se alcanzaron a actualizar los siguientes bloques:

4.4.1) Sólo bloque de directorio

R: (e) Se actualizaron los punteros del bloque de directorio, pero no el bloque índice al que apuntan, así que este contendrá cualquier cosa (consecuencia d). Por otro lado, como no queda marcado en el bitmap, podría darse el caso que a futuro se cree otro archivo y como bloque índice de este se emplee el mismo porque no fue marcado como ocupado, con lo que dos *file paths* llegarían al mismo bloque índice (problema c).

4.4.2) Bitmap y bloque de directorio

R: (d) Como el bloque índice se marca ocupado, no puede ocurrir que distintos *file paths* lleguen al mismo bloque índice, pero como este no fue escrito, va a contener cualquier cosa.

4.4.3) Bloque de directorio y bloque índice

R: (c) Dado que no se marca como ocupado, se podría dar el caso que, al crear otro archivo nuevo, se sobrescriba el bloque índice del actual. En este caso habrían dos *file paths* que llegan al mismo bloque índice.

5. Memoria principal

5.1 Suponga que se cuenta con un 90 % de TLB Hit Rate, el tiempo de acceso a la TLB es de 50 *ms*, el tiempo de acceso a memoria es de 200 *ms* y tiene un 99 % de Hit Rate, y el tiempo de acceso a disco (encontrar y traer la página a memoria) es de 1 *s*. ¿Cuánto es el tiempo de acceso promedio a una página?

R: Cuando hay un TLB Hit, existe un acceso a la TLB (50 ms) y un acceso a memoria a buscar la página que queremos utilizar (200 ms). Mientras que cuando tenemos una TLB Miss, accedemos a la TLB (50 ms), comprobamos que no existe la traducción que queremos, por lo tanto debemos ir a la tabla de páginas de la memoria a buscarla (200 ms), para finalmente cargar la página que usaremos (200 ms). En caso que la página este en disco debemos ir a buscar al disco la página (1 s) en vez de cargar la página desde memoria. Con esto el tiempo promedio es:

$$T = 0,9 \cdot (50 + 200)ms + 0,1 \cdot (0,99 \cdot (50 + 200 \cdot 2) + 0,01 \cdot (50 + 200 + 1000))ms$$

5.2 Dada una memoria física de 32 frames, direcciones físicas de 20 bits y una memoria virtual de 4 GB.

5.2.1) Determine el tamaño de la memoria física, la cantidad de páginas, el tamaño de los frames y el largo de una dirección virtual (en bits).

R: 1MB, 2^{17} pags, 32KB, 32 bits

5.2.2) Indique cual/es de las siguientes direcciones físicas pueden ser la traducción de la siguiente dirección virtual 0DEF4A49E:

- | | | | |
|----------|----------|----------|----------|
| a) 928DE | c) 1A49E | e) 610BA | g) 08874 |
| b) D331E | d) E939D | f) 02001 | h) 9249E |

R: El offset debe ser el mismo, por lo tanto debe ser cualquier dirección física que termine con 010 49E, en otras palabras que terminen con A49E o 249E. Entonces las únicas que cumplen con esto son 1A49E y 9249E

5.2.3) Asumiendo un disco con bloques direccionados con 24 bits, cuántos frames de esta memoria podemos guardar en un bloque al utilizar swapping ignorando los bits de metadata?

R: debido a los 24 bits para direccionar dentro de un bloque, estos son de 16MB. Al dividir el tamaño de un frame sobre esto quedamos con 2^9 frames en un bloque.

5.2.4) Asumiendo que el mismo disco guarda frames direccionando desde un bloque, asumiendo que se utilizan 23 bits de metadata, cuántas páginas podemos guardar con un bloque para direccionar todas las páginas que están en disco?

R: Considerando que los bloques son de 16MB y el número de página requiere 17 bits, cada entrada usa 40 bits, lo que corresponde a 5 bytes. Entonces podemos guardar $\frac{2^{24}}{5}$ frames.